

[← Go Back](#)

Hardware

Nano ESP32

Tutorials

[Arduino Nano ESP32 User Manual](#)[Arduino Nano ESP32 IoT Cloud Setup Guide](#)[Debugging with the Nano ESP32](#)[Device to Device Communication with ESP-NOW](#)[Getting Started with Nano ESP32](#)[Nano ESP32 Selecting Pin Configuration](#)[SPIFFS Partition on Nano ESP32](#)[Home](#) / [Hardware](#) / [Nano ESP32](#) / **SPIFFS Partition on Nano ESP32**

ON THIS PAGE

Hardware & Software Needed

[Set Up SPIFFS File System](#)[Example](#) +

SPIFFS Partition on Nano ESP32

Get started with the SPIFFS partition scheme on the Nano ESP32.

Author · [Hannes Siebeneicher](#) · Last revision · 12/18/2023

The SPIFFS (Serial Peripheral Interface Flash File System) is a file system designed to be used with NOR flash memory devices and microcontrollers that support the Serial Peripheral Interface (SPI) protocol such as the ESP32 found on the [Nano ESP32](#).

Before proceeding with the SPIFFS file system setup on Nano ESP, please be aware that the mounting instructions mentioned in a comment on the code from the packaged built-in example are tailored for specific older IDE versions such as IDE 1.8. These instructions might not be relevant to IDE 2.0, as it does not support the utilization of custom tools, which are unnecessary in this case.



Hardware & Software Needed

[Arduino Nano ESP32](#)[Arduino IDE](#)[Arduino ESP32 Board Package](#) installed (done in the Arduino IDE).[Help](#)

Set Up SPIFFS File System

Start by opening the **SPIFFS_Test example** by navigating to:

File > Examples > SPIFFS > SPIFFS_Test

 The example is also included at the bottom (see [Example](#)), together with an explanation of how the code work.



Open Example

Select the correct **Partition Scheme** by clicking:

Tools > Partition Scheme > With SPIFFS partition (advanced)



Select SPIFFS

Prepare your following these steps:

Take your **unplugged** board and connect a jumper cable between the **GND** and **B1** pins.

Plug it in and the RGB LED will turn on with a green or blue color.

While the **GND** and **B1** pins are shorted, press the white **RST**

Remove the jumper cable. The RGB LED should stay on, in a purple or yellow color.

Select the correct port:



Select Port

If your board shows up as a generic ESP32 device go to Tools

i > Board and select "Arduino Nano ESP32" as well as the correct port under Tools > Port.

Select the **Esptool as Programmer** by going to:

Tools > Programmer > Esptool



Select Programmer

Burn the bootloader by clicking:

Tools > Burn Bootloader

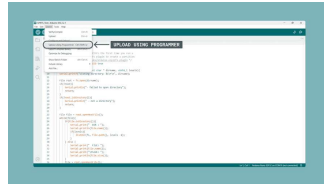


Burn Bootloader

We have now successfully changed the partition scheme and the final step is to upload a sketch using programmer

Upload the sketch via **Using Programmer** by going to:

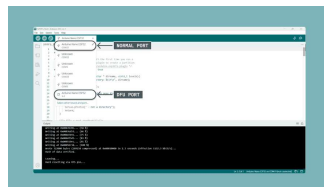
Sketch > Upload Using Programmer



Upload Using Programmer

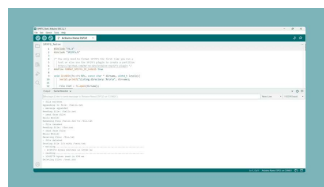
After the successful upload press the white **RST** button on the top of the board. You should see two ports in the drop-down menu, one showing a USB symbol and another one showing a cube.

Select the port next to the USB symbol.



Select the Correct Port

Open the serial monitor and you should see the following output:



Serial Output

i Note: It might take a short while for the messages to be printed. If you don't see any messages after one minute repeat the process.

As mentioned above the code is taken from **File > Examples > SPIFFS > SPIFFS_Test** and can also be found inside the IDE's example section. It showcases different features available when using the SPIFS file system.

```
1  #include "FS.h"
2  #include "SPIFFS.h"
3
4  /* You only need to for
5     test or else use the
6     https://github.com/m
7  #define FORMAT_SPIFFS_I
8
9  void listDir(fs::FS &fs
10     Serial.printf("List
11
12     File root = fs.open
13     if(!root){
14         Serial.println(
15         return;
16     }
17     if(!root.isDirectory
18         Serial.println(
19         return;
20     }
21
22     File file = root.op
23     while(file){
24         if(file.isDirec
25             Serial.prin
26             Serial.prin
27             if(levels){
28                 listDir
```

How It Works?

First, the **necessary libraries** are included:

```
1  #include "FS.h"
2  #include "SPIFFS.h"
```

The constant

SPIFFS file system should be formatted if it fails to mount. If it's `true`, the file system will be formatted:

```
1 #define FORMAT_SPIFFS_IF_FAILED
```

The code consists of several **functions** defined in the code showcasing **different features** such as:

`void listDir()` = List Files

`void readFile()` = Read Files

`void writeFile()` = Write Files

`void appendFile()` = Append a message to an existing file

`void renameFile()` = Rename Files

`void deleteFile()` = Delete Files

`void testFileIO()` = Testing a file's input and output by writing a large amount of data and then reading it.

Before calling all the functions mentioned above **two things** are done inside `setup()`:

- 1 The serial communication is initialized with a baud rate of 115200.
- 2 The SPIFFS file system is mounted and if `FORMAT_SPIFFS_IF_FAILED` is `true` it's formatted.

```
1 void setup(){
2   Serial.begin(115200);
3   if(!SPIFFS.begin(FORMAT
4     Serial.println("SPI
5     return;
6   }
7   ...
8 }
```

Lastly, all functions are called with parameters:

```
1 void setup(){
2   Serial.begin(115200);
3   if(!SPIFFS.begin(FORMA
4     Serial.println("SP
5     return;
6   }
7
8   listDir(SPIFFS, "/", 0
9   writeFile(SPIFFS, "/he
10  appendFile(SPIFFS, "/t
11  readFile(SPIFFS, "/he
12  renameFile(SPIFFS, "/t
13  readFile(SPIFFS, "/foc
14  deleteFile(SPIFFS, "/t
15  testFileIO(SPIFFS, "/t
16  deleteFile(SPIFFS, "/t
17  Serial.println( "Test
18 }
```

All messages printed on the serial monitor are defined in the respective function, except "Test Complete" which printed by `setup()` once everything is done.

Suggest changes

The content on docs.arduino.cc is facilitated

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#)

repository. If you see anything wrong, you can edit this page [here](#).

Was this article helpful?

